FACULTY OF ENGINEERING & TECHNOLOGY

ELECTRICAL AND COMPUTER ENGINEERING DEPARTMENT

ADVANCED DIGITAL DESIGN ENCS3310

COURSE PROJECT

Prepared by:

Name: Nagham Massis. **ID:1220149.**

Instructors: Abdellatif Abu-Issa.

Section: 3.

Date: 27/12/2024.

Brief Introduction:

The project focuses on the design and implementation of a structural comparator for both signed and unsigned 6-bit numbers. The primary objective is to create a circuit that accurately compares two input numbers and produces outputs indicating whether they are equal, greater, or smaller. This comparator will utilize a selection bit to determine the type of comparison (signed or unsigned) and will be constructed using a library of basic logic gates(Gate/ID 9).

Table of Contents:

Brief Introduction: I
Table of Figures: III
List of Tables: IV
Brief theoretical overview: 1
Design philosophy: 3
1. **The unsigned_comparator6bit Module** 3
2. **The signed_comparator6bit** 4
3. **The comparator6bit_1 Module(structural)** 5
4. **The comparator6bit_2 Module(behavioral)** 6
5. **Testbench: comparator6bit_tb** 7

Simulation Results: 9
Conclusion: 11

Table of Figures:

Figure 1: Symbols for AND, OR, NOT, NAND, NOR, XOR, and XNOR gates. 1
Figure 2: Result 1. 9
Figure 3: Result 2. 9
Figure 4: Error for the verification. 9
Figure 5: Another example on case 1. 10
Figure 6: Another example on case 2. 10

List of Tables:

Table 1: Comparator Output Conditions. 1
Table 2: Gate Delays. 2
Table 3: Some test cases for 6-bit Comparator. 2

Brief theoretical overview:

The project involves the design of a structural comparator, which is a component in digital systems used for comparing binary numbers. Comparators are essential in various applications, for example arithmetic operations.

1. **Comparator:** A comparator takes two binary inputs and determines their relational status. In this project, the comparator will handle both signed and unsigned numbers. For signed numbers, the 2's complement representation is used. The outputs of the comparator will indicate whether the first number (A) is equal to, greater than, or smaller than the second number (B).

Condition	Equal	Greater	Smaller
A = B	**1**	**0**	**0**
A > B	**0**	**1**	**0**
A < B	**0**	**0**	**1**

Table 1: Comparator Output Conditions.

2. **Logic Gates:** The design will be constructed using basic logic gates, including AND, OR, NOT, NAND, NOR, XOR, and XNOR gates. Each gate performs a specific logical function, and their combination allows for the creation of complex circuits. The project will utilize a library of gates, each with specified delays (Gate/ID 9), to ensure that the design meets timing requirements.

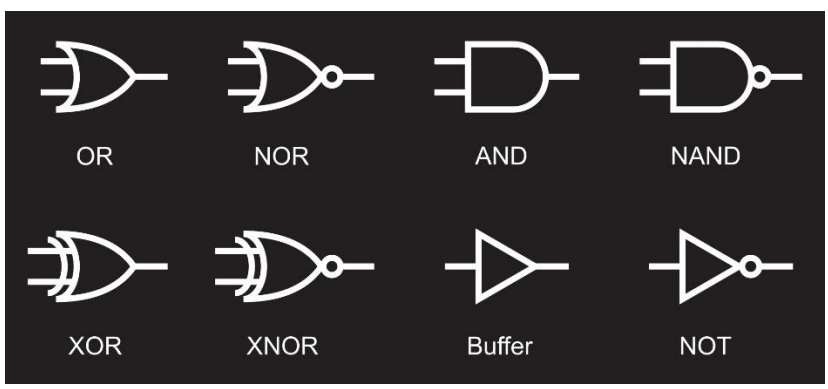

Figure 1: Symbols for AND, OR, NOT, NAND, NOR, XOR, and XNOR gates.

3. **Synchronous Design:** To make the comparator more reliable, registers will be added to its inputs and outputs, so it works with a clock signal. This ensures data is processed in a controlled and timed way.

Gate	Delay
NOT	**2**
NAND	**7**
NOR	**7**
AND	**8**
OR	**8**
XNOR	**9**
XOR	**10**

Table 2: Gate Delays.

4. **Verification and Testing**: A critical aspect of the project is the verification of the comparator's functionality. The design tested against all possible input combinations to ensure accurate outputs. Additionally, an intentional error introduced into the design to test the verification process.

A	B	$A > B$	$A < B$	$A = B$
000000 (0)	000000 (0)	0	0	1
000001 (1)	000000 (0)	1	0	0
000001 (1)	000001 (1)	0	0	1
000010 (2)	000001 (1)	1	0	0
111111 (63)	111110 (62)	1	0	0
111111 (63)	111111 (63)	0	0	1
111110 (62)	111111 (63)	0	1	0

Table 3: Some test cases for 6-bit Comparator.

Design philosophy:

The project involves designing a structural comparator for signed and unsigned 6-bit numbers. Each module in the design serves a specific function to achieve the overall goal of comparing two numbers.

1. **The unsigned_comparator6bit Module** is designed to compare two 6-bit unsigned binary numbers, A and B. The module determines whether A is greater than, less than, or equal to B, and generates the corresponding outputs: G (greater-than), L (less-than), and E (equal-to).

Inputs

- **A and B:** These are 6-bit wide inputs, representing the unsigned binary numbers to be compared.

Outputs

- **G (Greater-than):** This output is high (1) when A > B.
- **L (Less-than):** This output is high (1) when A < B.
- **E (Equal-to):** This output is high (1) when A == B.

Logic Components and Functionality

1. **NOT Gates:**
    - Used to invert the bits of A and B.
    - Delay of #2 is specified for the NOT gates (Gate/ID 9).
2. **XNOR Gates:**
    - Used to compare individual bits of A and B.
    - The XNOR operation produces a high (1) output when two input bits are equal.
    - Delay of #9 is specified for the XNOR gates (Gate/ID 9).
3. **AND Gate for Equality:**
    - Combines the equality results of all 6 bits (eq5 to eq0).
    - The output E is high (1) only when all bits of A and B are equal (A == B).
    - Delay of #8 is specified for the AND gate (Gate/ID 9).
4. **Greater-Than Condition:**
    - Calculated by examining the bits of A and B hierarchically, starting with the most significant bit (MSB):

        - If A[5] > B[5], the signal gt5 is high.
        - If A[5] == B[5], the next bit (A[4] vs B[4]) is checked, and so on.
    - An OR gate combines the individual gt signals to produce the G output, which is high (1) when A > B.
    - Delay of #8 is specified for the OR gate (Gate/ID 9).

5. **Less-Than Condition:**
    - Similar to the greater-than condition, calculated hierarchically:
        - If A[5] < B[5], the signal lt5 is high.
        - If A[5] == B[5], the next bit (A[4] vs B[4]) is checked, and so on.
    - An OR gate combines the individual lt signals to produce the L output, which is high (1) when A < B.
    - Delay of #8 is specified for the OR gate (Gate/ID 9).

Final Outputs

The final output signals are as follows:

- G is high (1) when A > B.
- L is high (1) when A < B.
- E is high (1) when A == B.

---

2. **The signed_comparator6bit Module** is designed to compare two 6-bit signed binary numbers, A and B, represented in 2's complement form. The module generates three output signals that represent the comparison results:

- G (Greater-than): High (1) when A > B.
- L (Less-than): High (1) when A < B.
- E (Equal-to): High (1) when A == B.

Handling Signed Numbers:

1. **Unsigned Numbers:** The MSB represents a magnitude bit.
2. **Signed Numbers (2's Complement):** The MSB acts as a sign bit:
    - 0: Represents a positive or zero value.
    - 1: Represents a negative value.

Logic Implementation

1. **Sign Bit Handling:**
    - Case 1:
        - If A[5] = 0 (positive) and B[5] = 1 (negative), then A > B.
        - The G (greater-than) output is set high.
    - Case 2:
        - If A[5] = 1 (negative) and B[5] = 0 (positive), then A < B.
        - The L (less-than) output is set high.
2. **Comparison of Magnitudes:**
    - When the sign bits of A and B are equal (A[5] == B[5]):
        - The comparison is based on the magnitude of the remaining bits (A[4:0] vs. B[4:0]).
        - This step follows the same logic as the unsigned_comparator6bit module, as the signs are consistent.

---

3. **The comparator6bit_1 Module(structural)** is a comparison unit that can handle both signed and unsigned comparisons of two 6-bit binary numbers (A and B). It uses a clock (clk) to synchronize operations and a select signal (S) to determine the type of comparison. This module outputs three signals: G (greater than), L (less than), and E (equal to). The outputs are based on whether A is greater than, less than, or equal to B.

Inputs

- **A and B**: 6-bit wide inputs representing the binary numbers to be compared.
- **S**: A single-bit signal that selects the comparison mode:
    - S = 1: Perform a **signed** comparison.
    - S = 0: Perform an **unsigned** comparison.
- **clk**: A clock signal to synchronize data processing.

Outputs

- **G (Greater-than)**: High when A > B.
- **L (Less-than)**: High when A < B.
- **E (Equal-to)**: High when A == B.

Registers and Wires

- **Internal Registers**:
    - in1 and in2: Store versions of A and B, respectively.
    - s: Stores version of the selection signal S.
- **Internal Wires**:
    - G_unsigned, L_unsigned, E_unsigned: Results of unsigned comparison from the unsigned_comparator6bit instance.
    - G_signed, L_signed, E_signed: Results of signed comparison from the signed_comparator6bit instance.

Components and Logic

1. **unsigned_comparator6bit Module**:
    - Compares in1 and in2 as **unsigned numbers**.
    - Produces outputs G_unsigned, L_unsigned, and E_unsigned.

2. **signed_comparator6bit Module**:
    - Compares in1 and in2 as **signed numbers** using 2's complement representation.
    - Produces outputs G_signed, L_signed, and E_signed.
3. **Clock Synchronization**:
    - On the positive edge of the clock:
        - Inputs A, B, and S are latched into in1, in2, and s, respectively.
        - This ensures stable and synchronized data for comparison.
4. **Comparison Mode Selection**:
    - The second always block determines the output based on the value of s:
        - **For S = 1 (Signed Comparison)**: Assigns the outputs of signed_comparator6bit to G, L, and E.
        - **For S = 0 (Unsigned Comparison)**: Assigns the outputs of unsigned_comparator6bit to G, L, and E.

---

4. **The comparator6bit_2 Module(behavioral)** is designed to compare two 6-bit binary numbers, A and B, based on their signed or unsigned nature. The comparison mode is determined by an input signal S. The module operates synchronously, updating its outputs on the positive edge of the clock (clk).

Inputs:
- A and B: 6-bit binary inputs to be compared.
- S: A single-bit control signal that selects the comparison mode:
    - S = 1: Perform signed comparison.
    - S = 0: Perform unsigned comparison.
- clk: Clock signal for synchronous operation.

Outputs:
- G (Greater-than): High (1) when A > B.
- L (Less-than): High (1) when A < B.
- E (Equal-to): High (1) when A == B.

Internal Registers:
- in1 and in2: Registered copies of inputs A and B.
- s: Registered copy of the control signal S.
- On the positive edge of the clock (clk), the inputs A, B, and S are stored in the internal registers in1, in2, and s. This ensures stable inputs for the comparison logic.

Comparison Logic:
- The module performs comparisons based on the value of the control signal s:
    - **Signed Comparison (s = 1)**:

    The $signed system function interprets the inputs in1 and in2 as signed numbers.
    The results are determined by comparing the signed values of in1 and in2.
    G: High if $signed(in1) > $signed(in2).
    L: High if $signed(in1) < $signed(in2).
    E: High if $signed(in1) == $signed(in2).

- **Unsigned Comparison (s = 0)**:

The inputs in1 and in2 are treated as unsigned binary numbers.
The results are determined by comparing the unsigned values of in1 and in2.
G: High if in1 > in2.
L: High if in1 < in2.
E: High if in1 == in2.

Synchronous Output Update:

- On each positive edge of the clock, the outputs G, L, and E are updated based on the comparison results.

---

5. **Testbench: comparator6bit_tb:** The testbench, comparator6bit_tb, verifies the functionality of the comparator6bit_1 and comparator6bit_2 modules by comparing their outputs (G, L, E) for all possible inputs of A and B in both signed and unsigned modes.

Inputs:

- A, B: Two 6-bit registers representing the numbers to be compared.
- S: Control signal to switch between signed (S = 1) and unsigned (S = 0) modes.
- clk: Clock signal for synchronous operation.

Outputs:

- G1, L1, E1: Outputs of comparator6bit_1.
- G2, L2, E2: Outputs of comparator6bit_2.

Modules Under Test:

- Two instances of the comparison modules:comparator6bit_1 and comparator6bit_2.
- Outputs of both modules are compared for correctness.

Clock Signal Generation:

- A periodic clock is generated using always #15 clk = ~clk, creating a clock signal with a period of 30 time units.

Test Procedure:

- **Initialization**:

  clk is initialized to 0.

  flag is initialized to 0 to track mismatches.

  S is initialized to 0 for unsigned mode.

  A and B are initialized to 6'b000000.

$monitor prints the values of A, B, S, and the outputs from both modules (G1, L1, E1, G2, L2, E2) at every time step.

- **Nested Loops**:

  Outer loop increments A and toggles the S signal after every 30 time units.

  Inner loop increments B while comparing the outputs of the two modules.

  A mismatch ({G1, L1, E1} !== {G2, L2, E2}) sets the flag to 1 and displays a mismatch error.

- **Pass/Fail Condition**:

  After all test cases, the value of flag determines the test result:

  flag = 0: All outputs matched, indicating the test passed.

  flag = 1: Mismatches were found, and the test failed.

- **Simulation Termination**:

  The simulation ends after the loops complete and test results are displayed using $finish.

Simulation Results:

- **Case 1: pass, all test vectors matched.**

Result:

```
# KERNEL: Time: 300 | clk: 0 | A: 000010 | B: 000110 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 315 | clk: 1 | A: 000010 | B: 000110 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 330 | clk: 0 | A: 000011 | B: 000110 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 345 | clk: 1 | A: 000011 | B: 000110 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 360 | clk: 0 | A: 000011 | B: 000110 | S: 1 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 375 | clk: 1 | A: 000011 | B: 000110 | S: 1 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 390 | clk: 0 | A: 000011 | B: 000111 | S: 1 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 405 | clk: 1 | A: 000011 | B: 000111 | S: 1 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 420 | clk: 0 | A: 000011 | B: 001000 | S: 1 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 435 | clk: 1 | A: 000011 | B: 001000 | S: 1 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: PASS: All test vectors matched.
```

Figure 2: Result 1.

Discussion:

At time 300, the clock (clk) is 0. After 15-time units, at time 315, the clock transitions to 1 (positive edge). Due to this positive edge, the registers capture the inputs: A = 000010 and B = 000110, and output the previous result. After an additional 30-time units, at time 345, the output is calculated, resulting in the comparison A < B, with the flags set as G1 = 0, L1 = 1, and E1 = 0. The test vectors match as expected.

- **Case 2: fail, mismatch found during testing.**

Result:

```
# KERNEL: Time: 330 | clk: 0 | A: 000011 | B: 000110 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 345 | clk: 1 | A: 000011 | B: 000110 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Mismatch at Time: 360 | A: 000011 | B: 000110 | S: 1 | G1: 0, L1: 0, E1: 0 | G2: 0, L2: 1, E2: 0
# KERNEL: Time: 360 | clk: 0 | A: 000011 | B: 000110 | S: 1 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 375 | clk: 1 | A: 000011 | B: 000110 | S: 1 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Mismatch at Time: 390 | A: 000011 | B: 000111 | S: 1 | G1: 0, L1: 0, E1: 0 | G2: 0, L2: 1, E2: 0
# KERNEL: Time: 390 | clk: 0 | A: 000011 | B: 000111 | S: 1 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 405 | clk: 1 | A: 000011 | B: 000111 | S: 1 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 420 | clk: 0 | A: 000011 | B: 001000 | S: 1 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: Time: 435 | clk: 1 | A: 000011 | B: 001000 | S: 1 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
# KERNEL: FAIL: Mismatches found during testing.
```

Figure 3: Result 2.

Discussion:

At time 330, the clock (clk) is 0. After 15-time units, at time 345, the clock transitions to 1 (positive edge). Due to this positive edge, the registers capture the inputs: A = 000011 and B = 000110, and output the previous result.

After an additional 30-time units, at time 375, the output is calculated, resulting in G1 = 0, L1 = 0, and E1 = 0, and G2 = 0, L2 = 1, and E2 = 0. This discrepancy is due to an error in the nor gate within the unsigned_comparator6bit used in the structural comparator6bit_1. The test vectors do not match as a result.

```
and #8 (lt1, nA1, B[1], eq5, eq4, eq3, eq2);
and #8 (lt0, nA0, B[0], eq5, eq4, eq3, eq2, eq1);
//or #8 (L, lt5, lt4, lt3, lt2, lt1, lt0);
nor #8 (L, lt5, lt4, lt3, lt2, lt1, lt0);
```

Figure 4: Error for the verification.

➢ Another examples:

```
◦ # KERNEL: Time: 126480 | clk: 0 | A: 000000 | B: 111000 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126495 | clk: 1 | A: 000000 | B: 111000 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126510 | clk: 0 | A: 000000 | B: 111001 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126525 | clk: 1 | A: 000000 | B: 111001 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126540 | clk: 0 | A: 000000 | B: 111010 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126555 | clk: 1 | A: 000000 | B: 111010 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126570 | clk: 0 | A: 000000 | B: 111011 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126585 | clk: 1 | A: 000000 | B: 111011 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126600 | clk: 0 | A: 000000 | B: 111100 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126615 | clk: 1 | A: 000000 | B: 111100 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126630 | clk: 0 | A: 000000 | B: 111101 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126645 | clk: 1 | A: 000000 | B: 111101 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126660 | clk: 0 | A: 000000 | B: 111110 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126675 | clk: 1 | A: 000000 | B: 111110 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126690 | clk: 0 | A: 000000 | B: 111111 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126705 | clk: 1 | A: 000000 | B: 111111 | S: 0 | G1: 0 | L1: 1 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: PASS: All test vectors matched.
```

Figure 5: Another example on case 1.

```
◦ # KERNEL: Time: 126570 | clk: 0 | A: 000000 | B: 111011 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126585 | clk: 1 | A: 000000 | B: 111011 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Mismatch at Time: 126600 | A: 000000 | B: 111100 | S: 0 | G1: 0, L1: 0, E1: 0 | G2: 0, L2: 1, E2: 0
◦ # KERNEL: Time: 126600 | clk: 0 | A: 000000 | B: 111100 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126615 | clk: 1 | A: 000000 | B: 111100 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Mismatch at Time: 126630 | A: 000000 | B: 111101 | S: 0 | G1: 0, L1: 0, E1: 0 | G2: 0, L2: 1, E2: 0
◦ # KERNEL: Time: 126630 | clk: 0 | A: 000000 | B: 111101 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126645 | clk: 1 | A: 000000 | B: 111101 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Mismatch at Time: 126660 | A: 000000 | B: 111110 | S: 0 | G1: 0, L1: 0, E1: 0 | G2: 0, L2: 1, E2: 0
◦ # KERNEL: Time: 126660 | clk: 0 | A: 000000 | B: 111110 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126675 | clk: 1 | A: 000000 | B: 111110 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Mismatch at Time: 126690 | A: 000000 | B: 111111 | S: 0 | G1: 0, L1: 0, E1: 0 | G2: 0, L2: 1, E2: 0
◦ # KERNEL: Time: 126690 | clk: 0 | A: 000000 | B: 111111 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: Time: 126705 | clk: 1 | A: 000000 | B: 111111 | S: 0 | G1: 0 | L1: 0 | E1: 0 | G2: 0 | L2: 1 | E2: 0
◦ # KERNEL: FAIL: Mismatches found during testing.
```

Figure 6: Another example on case 2.

Conclusion:

In summary, this project successfully created a structural comparator for comparing signed and unsigned 6-bit numbers. The design accurately identifies when the numbers are equal, greater, or smaller, handling both types of number formats signed and unsigned numbers correctly. By adding registers and a clock signal, the circuit operates in a synchronized manner, which improves its reliability. The project also included a way to detect errors, showing that the system can identify problems effectively. Also, we implement a behavioral comparator to verify that the system work properly. The modular design makes the project easy to understand and maintain, and thorough testing confirmed that it works correctly for all input combinations.